

A Significance-Driven Hybrid Approximate 8×8 Multiplier for LUT-Constrained FPGAs in Error-Tolerant Image Processing

Student Name 1, Student Name 2, Student Name 3
Department of Electronics and Communication Engineering
College Name, City, India
student1@college.edu

Guide Name
Department of Electronics and Communication Engineering
College Name, City, India
guide@college.edu

Abstract—Multipliers dominate the logic cost of image-processing and machine-learning datapaths on FPGAs, yet many such workloads tolerate small arithmetic errors. This paper presents a family of recursive 8×8 unsigned multipliers, built from 2×2 blocks, in which approximation is applied selectively to the least-significant partial products. Five approximate configurations are compared against an exact reference: Kulkarni-style under-designed 2×2 blocks applied at three significance levels, truncation of the lowest 4×4 partial product, and a hybrid (HYB) that combines truncation of the lowest block with approximate middle blocks. All designs were verified exhaustively over the 65,536 input pairs and synthesised for the Xilinx 7-series (Artix-7) fabric. The hybrid design uses 80 LUTs instead of 112 (a 28.6% reduction) with a normalised mean error distance of 0.24%. In three image-processing applications driven by the RTL product tables, it keeps PSNR between 46.7 and 50.1 dB and SSIM at or above 0.996. The exploration also shows that truncating the lowest partial product is Pareto-superior to applying the under-designed 2×2 block in the upper partial products at the LUT granularity of the target fabric.

Index Terms—approximate computing, approximate multiplier, FPGA, Verilog, LUT optimisation, image processing

I. INTRODUCTION

Approximate computing trades a controlled loss of accuracy for savings in area, delay and energy in applications whose outputs are judged by humans or by statistical quality metrics [1], [2]. Multipliers are a natural target, because they are the most expensive arithmetic operator in filters, transforms and neural-network layers [3].

Most approximate multipliers were designed for ASICs, where savings are counted in transistors or gates. On an FPGA the cost unit is different: logic is implemented in 6-input look-up tables (LUTs) and dedicated carry chains [4]. A simplification that removes a gate does not necessarily remove a LUT, which is why FPGA-specific approximate multipliers have been proposed [5], [6]. Even for exact arithmetic, a hand-structured 4-bit multiplier can use fewer LUTs than the netlist inferred from a behavioural $a * b$ [7].

This paper asks a simple, practical question: *where* in a recursive multiplier should approximation be placed to obtain the best LUT-versus-accuracy trade-off on a low-cost FPGA? The contributions are:

- A parameterised Verilog family of six 8×8 multipliers (one exact, five approximate) built from 2×2 blocks, with approximation placed by partial-product significance.
- Exhaustive functional verification and error characterisation (error rate, MED, NMED, MRED, worst-case error) over all 65,536 input pairs.
- A synthesis study for the Xilinx 7-series fabric used on the Basys-3 teaching board [8], showing that a hybrid of truncation and approximation (HYB) saves 28.6% of LUTs.
- An application study (image multiplication, Gaussian smoothing and image blending) in which every multiplication is taken from the RTL simulation output.

II. BACKGROUND AND RELATED WORK

A. Recursive multiplication

An $n \times n$ product can be decomposed into four $n/2 \times n/2$ partial products. With $A = \{A_H, A_L\}$ and $B = \{B_H, B_L\}$ split into 4-bit halves,

$$AB = 2^8 A_H B_H + 2^4 (A_H B_L + A_L B_H) + A_L B_L. \quad (1)$$

Each 4×4 term is decomposed again into 2×2 terms with weights 2^4 , 2^2 and 2^0 . The weights in (1) show that an error in $A_L B_L$ is scaled by 2^0 , whereas an error in $A_H B_H$ is scaled by 2^8 .

B. Under-designed 2×2 multiplier

Kulkarni *et al.* [9] observed that an exact 2×2 multiplier needs a fourth output bit only for $3 \times 3 = 9$. Their approximate block returns 7 for that single input pair and is exact otherwise, so its output fits in three bits:

$$p_0 = a_0 b_0, \quad p_1 = a_1 b_0 \vee a_0 b_1, \quad p_2 = a_1 b_1. \quad (2)$$

One of the 16 input combinations is wrong, and the result always under-estimates the true product.

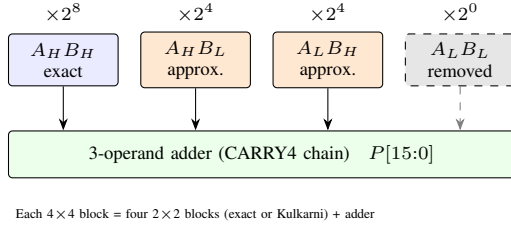


Fig. 1. Partial-product structure of the 8×8 multiplier, shown for the proposed hybrid (HYB) configuration. Approximation is applied only to the lower-weight partial products.

C. FPGA-oriented approximate multipliers

Architectural-space exploration of approximate multipliers [10] and FPGA-specific designs that exploit LUT and carry-chain structure [5] show that FPGA savings differ from ASIC savings. DyRecMul uses dynamically reconfigurable LUTs to cut LUT usage by 64% for signed 8-bit multiplication on an UltraScale+ device [6], and hand-optimised exact small multipliers can beat synthesis-inferred ones [7]. This work is deliberately simpler: it uses only standard Verilog and a significance-driven placement rule, so the designs can be reproduced with open-source tools and a low-cost board.

III. PROPOSED MULTIPLIER FAMILY

A. Design space

Fig. 1 shows the partial-product structure. A single Verilog module, `mul8x8`, takes a `MODE` parameter that sets, for each 4×4 partial product, whether it is exact, built from approximate 2×2 blocks, or removed:

- **EXACT**: all blocks exact (reference).
- **AM-L**: $A_L B_L$ approximate.
- **AM-LM**: $A_L B_L$, $A_H B_L$ and $A_L B_H$ approximate.
- **AM-ALL**: all four blocks approximate [9].
- **TRUNC-L**: $A_L B_L$ removed (forced to zero).
- **HYB** (proposed): $A_L B_L$ removed and $A_H B_L$, $A_L B_H$ approximate; $A_H B_H$ exact.

The rule is significance-driven: approximation is added from the lowest weight upwards, and the most significant block $A_H B_H$ is kept exact in every design except AM-ALL. Because every variant only removes or under-estimates partial products, all approximate products satisfy $\tilde{P} \leq AB$. This property was checked exhaustively, and it makes the error one-sided and easy to compensate if needed.

B. Implementation

The RTL uses three modules (`mul2x2`, `mul4x4`, `mul8x8`) with `generate` blocks selecting exact or approximate sub-blocks. A registered wrapper (`mul8x8_pipe`) places the multiplier between input and output registers for clocked use. A Basys-3 top level instantiates all six multipliers: switches provide the operands, LEDs show the product, push-buttons select the mode, and the seven-segment display shows the mode and the error magnitude $|AB - \tilde{P}|$.

IV. EVALUATION METHODOLOGY

Functional verification. A self-checking Icarus Verilog [11] testbench applies all 256×256 input pairs to every design. It checks that the exact recursive design and the behavioural `a*b` both equal the reference product, and that no approximate design over-estimates it. All 65,536 products of every design are written to a file. An independent Python model of each design reproduces the RTL output bit for bit.

Error metrics. With error distance $ED = |AB - \tilde{P}|$ and $P_{\max} = 255^2$, we report the error rate (ER, fraction of inputs with $ED > 0$), mean error distance (MED), normalised MED (NMED = $MED/P_{\max}$), mean relative error distance (MRED, over non-zero products) and worst-case error (WCE) [3].

Synthesis. Designs were synthesised with Yosys [12] (`synth_xilinx -family xc7 -nodsp`) for the 7-series fabric of the Artix-7 XC7A35T on the Basys-3 board [8]. DSP inference was disabled so that all designs are compared in LUTs and CARRY4 cells [4]. The primary flow preserves hierarchy, so each block is mapped as designed. A fully flattened ABC9 flow is reported as a sensitivity check. Logic depth is the longest path in mapped cells, excluding I/O buffers.

Applications. Three 8-bit kernels were run on standard 512×512 greyscale test images from scikit-image [13], using only the RTL product tables: (i) pixel-wise image multiplication, ($I_1 I_2 \gg 8$); (ii) 3×3 Gaussian smoothing with integer weights $\{20, 32, 48\}$ summing to 256 [14]; and (iii) alpha blending with $\alpha = 179/255$. Quality is measured against the exact output with PSNR and SSIM [15].

V. RESULTS

A. Functional verification and waveforms

The exhaustive testbench reported zero mismatches for the exact designs and zero over-estimations for the approximate designs. Fig. 2 shows the clocked simulation at 100 MHz. Products appear two cycles after their operands. The 3×3 case returns 7 in the three AM designs. TRUNC-L and HYB return 0 for any pair whose upper nibbles are both zero (e.g. 12×10 , 3×3 , 15×15), because the whole product then lies in the removed $A_L B_L$ term. The 255×255 case shows the error growing with the number of approximated blocks.

B. Error characteristics

Table I lists the error metrics. Approximating only $A_L B_L$ (AM-L) produces a very small NMED of 0.0048%, while applying the under-designed block everywhere (AM-ALL) raises the worst-case error to 22.2% of full scale. TRUNC-L has a high error rate (87.9%) but a bounded worst-case error of 225, because the removed partial product is at most 15×15 . HYB inherits that bound for its lowest block and adds the small middle-block error, reaching NMED 0.24%.

C. FPGA resources

Table II gives the synthesis results. In the hierarchy-preserving flow, the exact recursive design uses 112 LUTs, practically the same as the behavioural `a*b` (113 LUTs plus

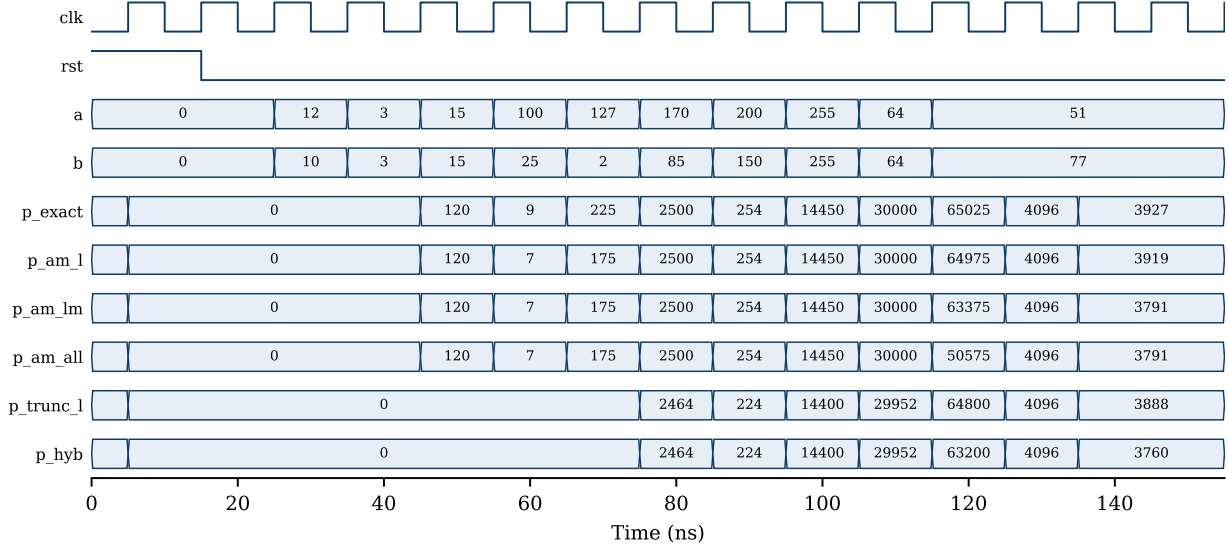


Fig. 2. Simulation waveform (Icarus Verilog, 100MHz clock). Products appear two cycles after the operands. For $a = b = 255$ the outputs are 65025 (EXACT), 64975 (AM-L), 63375 (AM-LM), 50575 (AM-ALL), 64800 (TRUNC-L) and 63200 (HYB).

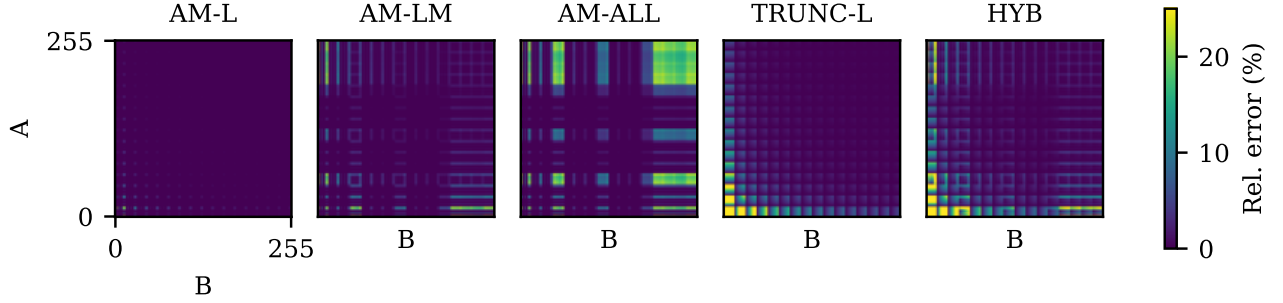


Fig. 3. Relative error over the operand space (A vertical, B horizontal; colour scale clipped at 25%). For TRUNC-L and HYB the largest relative errors occur at small operands, where the removed $A_L B_L$ term is a large fraction of the product. AM-ALL also errs at large operands, because its most significant block $A_H B_H$ is approximated.

TABLE I
ERROR METRICS OVER ALL 65,536 INPUT PAIRS

Design	ER (%)	MED	NMED (%)	MRED (%)	WCE
EXACT	0.00	0.00	0.0000	0.000	0
AM-L	19.14	3.12	0.0048	0.076	50
AM-LM	40.67	103.12	0.1586	0.925	1650
AM-ALL	46.73	903.12	1.3889	3.280	14450
TRUNC-L	87.89	56.25	0.0865	1.859	225
HYB	90.28	156.25	0.2403	2.708	1825

TABLE II
SYNTHESIS RESULTS, XILINX 7-SERIES (YOSYS, NO DSP)

Design	Hierarchical (primary)				Flat
	LUT	CARRY4	Depth	Saving	LUT
Behavioural $a \times b$	113 [†]	4	9	-0.9%	103
EXACT	112	12	8	-	103
AM-L	108	12	8	3.6%	120
AM-LM	100	12	8	10.7%	97
AM-ALL	96	12	8	14.3%	88
TRUNC-L	88	10	7	21.4%	82
HYB	80	10	7	28.6%	62

[†]plus 22 MUXF7 and 1 MUXF8; with DSP inference: 1 DSP48E1.

23 wide-mux cells). Each approximate 2×2 block saves one LUT because its fourth output is constant, so AM-L, AM-LM and AM-ALL save 4, 12 and 16 LUTs. Removing $A_L B_L$ saves a whole 4×4 block and two CARRY4 cells, and shortens the longest path by one cell. HYB combines both effects: 80 LUTs, a 28.6% reduction.

The flattened ABC9 flow lets the tool re-optimize across

block boundaries. It confirms the ordering for the most aggressive designs (HYB: 62 LUTs versus 103 for EXACT, a 39.8% reduction), but it is not monotonic for the lightest approximation: AM-L grows to 120 LUTs. Savings from small

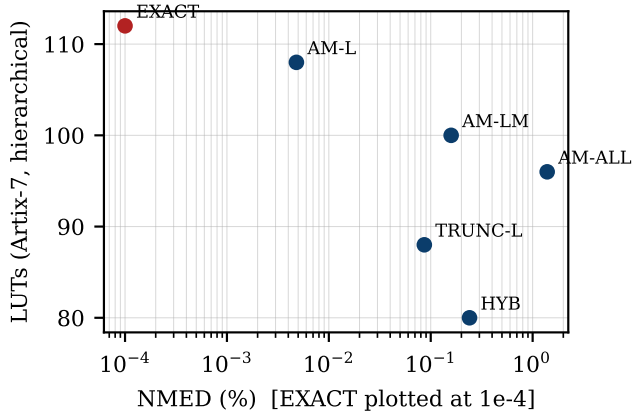


Fig. 4. LUT count versus NMED. EXACT is plotted at 10^{-4} for the log axis. EXACT, AM-L, TRUNC-L and HYB form the Pareto front.

TABLE III
IMAGE QUALITY VERSUS EXACT OUTPUT: PSNR (dB) / SSIM

Design	Multiplication	Gaussian	Blending
AM-L	64.81 / 0.9998	identical	60.93 / 0.9996
AM-LM	51.99 / 0.9973	56.17 / 0.9988	49.38 / 0.9980
AM-ALL	35.27 / 0.9851	34.98 / 0.9959	34.16 / 0.9951
TRUNC-L	55.70 / 0.9984	51.50 / 0.9976	51.99 / 0.9980
HYB	50.12 / 0.9960	49.71 / 0.9972	46.66 / 0.9975

approximations are therefore sensitive to synthesis heuristics, whereas the structural savings of truncation and HYB hold in both flows.

D. Accuracy–area trade-off

Fig. 4 plots LUTs against NMED. TRUNC-L is both smaller and more accurate than AM-LM and AM-ALL, so applying the under-designed block to the middle and upper partial products is dominated by simply removing the lowest one. The Pareto front is formed by EXACT, AM-L, TRUNC-L and HYB. HYB gives the lowest LUT count for NMED below 0.25%.

E. Image-processing quality

Table III and Fig. 5 report the application results. All designs except AM-ALL keep PSNR above 46 dB and SSIM of at least 0.996, a level at which differences are not visible. AM-L is bit-identical to EXACT in Gaussian smoothing, because an AM-L error needs a 2-bit field equal to 3 in the low nibble of both operands, and the low nibbles of the kernel weights (20, 32, 48) never contain one. AM-ALL drops to about 34–35 dB, which is consistent with its much larger worst-case error.

F. Discussion and limitations

The results support a simple design rule for LUT-based FPGAs: remove the lowest-weight partial product first, and approximate the middle ones only if more area must be saved. Two limitations should be noted. First, resources were obtained

with an open-source synthesis flow. Vendor place-and-route (Vivado) timing and power figures for the Basys-3 board are needed to quantify delay and energy, and absolute LUT counts will differ from Vivado’s. Second, TRUNC-L and HYB return zero when both operands are below 16, so they suit data dominated by mid-to-high values (such as natural images) but not kernels that multiply many small numbers. Their error is also one-sided (biased low), and a constant compensation term could reduce MED further at negligible cost.

VI. CONCLUSION

A parameterised family of recursive 8×8 approximate multipliers was designed in Verilog, verified exhaustively, synthesised for the Xilinx 7-series fabric and evaluated in three image-processing kernels. Significance-driven placement matters: removing the least-significant 4×4 partial product beats applying the under-designed 2×2 block in higher partial products. Combining the two in the proposed HYB design saves 28.6% of LUTs (39.8% in a flattened flow) with NMED 0.24% and image PSNR of at least 46.7 dB. Future work includes Vivado implementation with timing and power measurement on the Basys-3 board, bias compensation, and integration into a convolution accelerator.

ACKNOWLEDGMENT

The authors thank the Department of Electronics and Communication Engineering, College Name, for laboratory support. Parts of this manuscript and its Verilog code were drafted with the assistance of an AI tool; the authors reviewed, verified and take responsibility for all content.

REFERENCES

- [1] J. Han and M. Orshansky, “Approximate computing: An emerging paradigm for energy-efficient design,” in *Proc. 18th IEEE European Test Symposium (ETS)*, 2013.
- [2] S. Mittal, “A survey of techniques for approximate computing,” *ACM Computing Surveys*, vol. 48, no. 4, pp. 62:1–62:33, 2016.
- [3] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, “Approximate arithmetic circuits: A survey, characterization, and recent applications,” *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.
- [4] *7 Series FPGAs Configurable Logic Block User Guide (UG474)*, Xilinx (AMD).
- [5] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, “High-performance accurate and approximate multipliers for FPGA-based hardware accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 211–224, 2022.
- [6] S. Vakili, M. Vaziri, A. Zarei, and J. M. P. Langlois, “DyRecMul: Fast and low-cost approximate multiplier for FPGAs using dynamic reconfiguration,” *ACM Transactions on Reconfigurable Technology and Systems*, 2024, doi: 10.1145/3663480; preprint arXiv:2310.10053.
- [7] Kida and Sato, “Hardware-efficient accurate 4-bit multiplier for Xilinx 7 series FPGAs,” arXiv:2510.21533, 2025.
- [8] *Basys 3 FPGA Board Reference Manual*, Digilent Inc.
- [9] P. Kulkarni, P. Gupta, and M. Ercegovac, “Trading accuracy for power with an underdesigned multiplier architecture,” in *Proc. 24th International Conference on VLSI Design*, 2011, pp. 346–351.
- [10] S. Rehman, W. El-Harouni, M. Shafique, A. Kumar, and J. Henkel, “Architectural-space exploration of approximate multipliers,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016.
- [11] S. Williams, “Icarus Verilog,” <https://steveicarus.github.io/iverilog/>.
- [12] C. Wolf, “Yosys open SYnthesis suite,” <https://yosyshq.net/yosys/>.

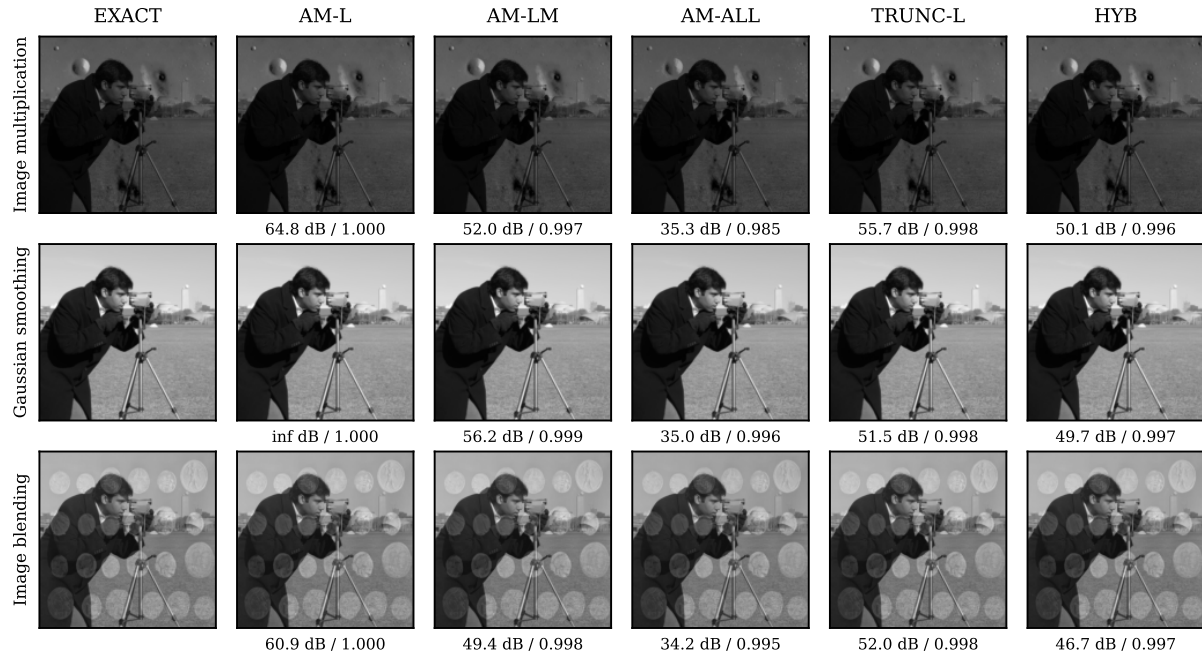


Fig. 5. Outputs of the three image kernels computed with the RTL product tables of each multiplier. Labels give PSNR / SSIM against the exact output.

- [13] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.
- [14] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.